



Hadoop MapReduce(suite et fin)

Exemple : Map-Reduce wordcount

- Créer un fichier nommé « data.txt » contenant des données dans le lecteur C.
- Créer un repertoire nommé « input_dir » dans HDFS :
`hadoop fs -mkdir /input_dir`
- Copier le fichier « data.txt » dans le repertoire « input_dir » de HDFS :
`hadoop fs -put C:/data.txt /input_dir`
- Visualiser les droits sur le fichier « data.txt » dans HDFS :
`hadoop dfs -ls /input_dir/data.txt`
Ou `hdfs dfs -ls /input_dir/data.txt`
- Visualiser en console le fichier « data.txt » dans HDFS :
`hadoop fs -cat /input_dir/data.txt`
- Executer mapReduce sur le fichier « data.txt » de HDFS :
`hadoop jar C:/hadoop-3.3.1/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.1.jar wordcount /input_dir /output_dir`
- Visualiser le resultat dans HDFS :
`hadoop dfs -ls / output_dir`
Puis afficher le contenu
`Hadoop dfs -cat / output_dir /part-*`

NB : Il arrive que le Job mapReduce prenne du temps lors de l'exécution du programme du wordcount, alors il faut donc définir les propriétés suivantes dans le fichier mapred-site.xml pour résoudre le problème :

```
<property>
<name>yarn.app.mapreduce.am.env</name>
<value>HADOOP_MAPRED_HOME=$HADOOP_HOME</value>
</property>
<property>
<name>mapreduce.map.env</name>
<value>HADOOP_MAPRED_HOME=$HADOOP_HOME</value>
</property>
<property>
<name>mapreduce.reduce.env</name>
<value>HADOOP_MAPRED_HOME=$HADOOP_HOME</value>
</property>
```

Où \$HADOOP_HOME => /C:/hadoop-3.3.1

Puis arrêter : stop-all.cmd et démarrer : start-all.cmd , et executer

Hadoop Streaming :

Utilisation de Python avec Hadoop

Exemple : Map-Reduce wordcount

Sur Pycharm éditer les fichiers mapper.py et reducer.py :

- Le fichier mapper.py :

```
#!/usr/bin/env python
import sys
# L'entrée est STDIN (input standard)
for line in sys.stdin:
    #Supprimer les espaces blancs de début et de fin
    line = line.strip()
    # Split : Diviser la ligne en mots et retourner une liste de mots
    words = line.split()

    #augmenter les compteurs
    for word in words:
        #Ecrire les résultats sur STDOUT(standard output)
        #ce que nous produisons ici sera l'entrée pour l'étape de reduce,
        c'est-à-dire l'entrée pour reducer.py
        #tab-delimited ; le nombre de mots trivial est 1
        print("{}\t{}".format(word, 1))
```

- Le fichier reducer.py :

```
#!/usr/bin/env python

from operator import getitem
import sys

current_word = {}
# L'entrée STDIN
for line in sys.stdin:

    # Supprimer les espaces blancs de début et de fin
    line = line.strip()

    #analyser l'entrée que nous avons obtenue de mapper.py
    word, count = line.split('\t',1)

    #convertir count (actuellement une chaîne) en entier
    try:
        count = int(count)

    except ValueError:
        #cas où count n'était pas un entier, ignorer cette ligne
        continue
    try:
        current_word[word] += count
    except:
        current_word[word] = count

#Afficher les mots et leur occurrence
for word in current_word.keys():
    print("{}\t{}".format (word, current_word[word]))
```

Sur Pycharm accéder au dossier contenant les fichiers mapper.py et reducer.py et tapez en console :

```
cat data.txt | python mapper.py | python reducer.py
type data.txt | python mapper.py | python reducer.py
```

où | => pipe (Alt gr +6) et cat => type (en windows)

Sur DOS accéder au dossier contenant les fichiers mapper.py et reducer.py et tapez en console :

```
type data.txt | python mapper.py | python reducer.py
```

Avec Hadoop streaming tapez en console :

```
Hadoop jar C:\hadoop-3.3.1\share\hadoop\tools\lib\hadoop-streaming-3.3.1.jar
-file emplacement_windows\mapper.py
-file emplacement_windows\reducer.py
-mapper "python mapper.py" -reducer "python reducer.py"
-input /input_dir/data.txt -output /sortie_out
```

Visualiser le resultat :

hadoop dfs -ls /sortie_out

Puis afficher le contenu

Hadoop dfs -cat /sortie_out/part-*